

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №4
по дисциплине «Базы данных»
Вариант №3154

Выполнил:

Дядев Владислав Александрович

Группа Р3131

Проверил:

Вербовой А. А.

Санкт-Петербург

2025

Содержание

Задание.....	3
Порядок выполнения.....	5
Запрос №1	5
Индексы	6
Результат EXPLAIN ANALYSE	6
Запрос №2	7
Индексы	7
Результат EXPLAIN ANALYSE	8
Вывод	9

Задание

Вариант №3154

Составить запросы на языке SQL (пункты 1–2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1–2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.

Изменяются ли планы при добавлении индекса и как?

Для запросов 1–2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_СЕССИЯ.ИД.

Фильтры (AND):

а) Н_ЛЮДИ.ОТЧЕСТВО < Георгиевич.

б) Н_СЕССИЯ.ИД > 32199.

с) Н_СЕССИЯ.ИД > 32199.

Вид соединения: LEFT JOIN.

2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.ЧЛВК_ИД,
Н_УЧЕНИКИ.ИД.

Фильтры: (AND)

а) Н_ЛЮДИ.ИД > 100012.

б) Н_ОБУЧЕНИЯ.НЗК = 933232.

Вид соединения: LEFT JOIN.

Порядок выполнения

Запрос №1

Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.

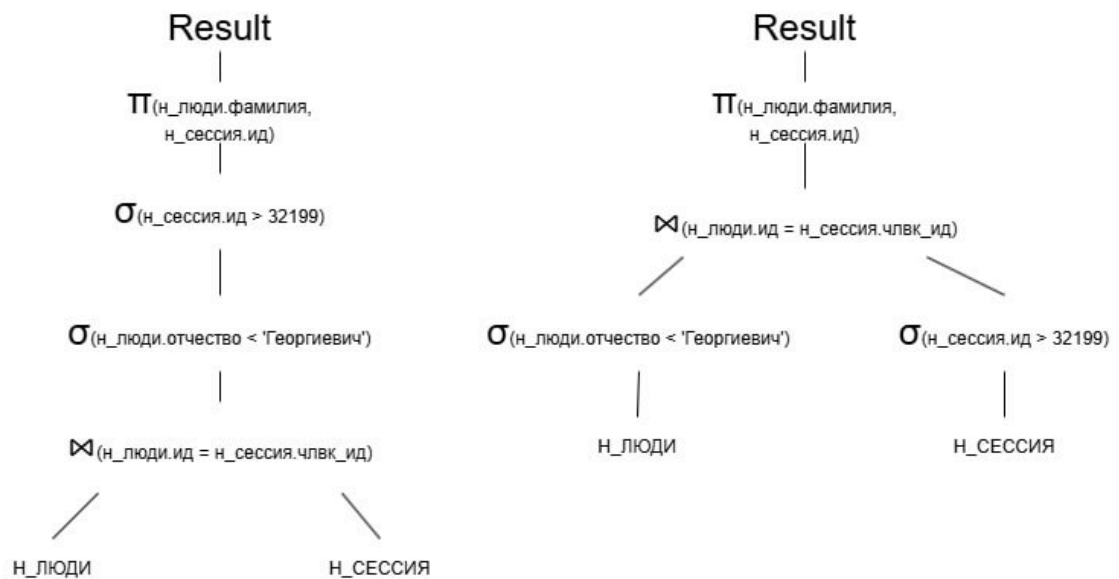
Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_СЕССИЯ.ИД.

Фильтры (AND):

- а) Н_ЛЮДИ.ОТЧЕСТВО < Георгиевич.
- б) Н_СЕССИЯ.ИД > 32199.
- с) Н_СЕССИЯ.ИД > 32199

Вид соединения: LEFT JOIN.

```
SELECT Н_ЛЮДИ.ФАМИЛИЯ, Н_СЕССИЯ.ИД  
FROM Н_ЛЮДИ  
LEFT JOIN Н_СЕССИЯ ON Н_ЛЮДИ.ИД = Н_СЕССИЯ.ЧЛВК_ИД  
WHERE Н_ЛЮДИ.ОТЧЕСТВО < 'Георгиевич'  
AND Н_СЕССИЯ.ИД > 32199;
```



Оптимальным является план №2, так как он производит объединение таблиц по ранее выбранным атрибутам, а не по таблицам целиком.

Индексы

```
CREATE INDEX "ИНДЕКС_ЛЮДИ_ИД" ON "Н_ЛЮДИ" USING hash("ИД");  
CREATE INDEX "ИНДЕКС_СЕССИЯ_ЧЛВК_ИД" ON "Н_СЕССИЯ" USING hash("ЧЛВК_ИД");  
CREATE INDEX "ИНДЕКС_ЛЮДИ_ОТЧЕСТВО" ON "Н_ЛЮДИ" USING btree("ОТЧЕСТВО");  
CREATE INDEX "ИНДЕКС_СЕССИЯ_ИД" ON "Н_СЕССИЯ" USING btree("ИД");
```

Добавление этих индексов должно ускорить выполнение запросов, так как по перечисленным полям происходит выборка с использованием оператора сравнения. Так же быстрее будет происходить соединение таблиц. В последних двух случаях используются операторы сравнения „>“ и „<“, так что эффективнее использовать btree. В первых двух случаях используется прямое сравнение, так что эффективнее использовать хэш-индекс.

При добавлении индексов планы выполнения запросов изменятся, так как будет происходить индексный скан и Nested Loop станет быстрее благодаря индексам.

Результат EXPLAIN ANALYSE

```
Nested Loop (cost=0.28..126.21 rows=1 width=20) (actual time=0.395..0.396 rows=0 loops=1)  
  -> Seq Scan on "Н_СЕССИЯ" (cost=0.00..117.90 rows=1 width=8) (actual time=0.394..0.395  
rows=0 loops=1)  
    Filter: ("ИД" > 32199)  
    Rows Removed by Filter: 3752  
  -> Index Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" (cost=0.28..8.30 rows=1 width=20) (never  
executed)  
    Index Cond: ("ИД" = "Н_СЕССИЯ"."ЧЛВК_ИД")  
    Filter: (("ОТЧЕСТВО")::text < 'Георгиевич'::text)  
Planning Time: 1.272 ms  
Execution Time: 0.455 ms
```

Запрос №2

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.ИД.

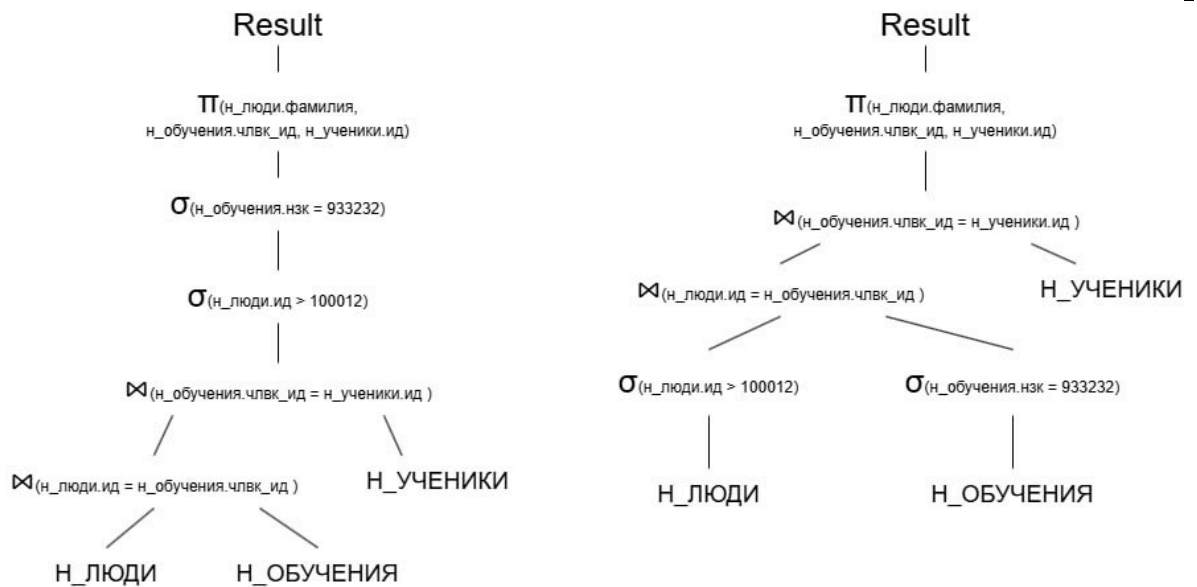
Фильтры: (AND)

a) Н_ЛЮДИ.ИД > 100012.

b) Н_ОБУЧЕНИЯ.НЗК = 933232.

Вид соединения: LEFT JOIN.

```
SELECT Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.ИД
FROM Н_ЛЮДИ
LEFT JOIN Н_ОБУЧЕНИЯ ON Н_ЛЮДИ.ИД = Н_ОБУЧЕНИЯ.ЧЛВК_ИД
LEFT JOIN Н_УЧЕНИКИ ON Н_УЧЕНИКИ.ИД = Н_ОБУЧЕНИЯ.ЧЛВК_ИД
WHERE Н_ЛЮДИ.ИД > 100012
AND Н_ОБУЧЕНИЯ.НЗК = '933232';
```



Оптимальным является план №2, так как он производит объединение таблиц по ранее выбранным атрибутам, а не по таблицам целиком.

Индексы

```
CREATE INDEX "ИНДЕКС_ЛЮДИ_ИД" ON "Н_ЛЮДИ" USING hash("ИД");
CREATE INDEX "ИНДЕКС_ОБУЧЕНИЯ_ЧЛВК_ИД" ON "Н_ОБУЧЕНИЯ" USING hash("ЧЛВК_ИД");
CREATE INDEX "ИНДЕКС_УЧЕНИКИ_ИД" ON "Н_УЧЕНИКИ" USING hash("ИД");
CREATE INDEX "ИНДЕКС_ОБУЧЕНИЯ_НЗК" ON "Н_ОБУЧЕНИЯ" USING hash("НЗК");
```

Добавление этих индексов должно ускорить выполнение запросов, так как по перечисленным полям происходит выборка с использованием оператора сравнения. Так же быстрее будет происходить соединение таблиц. Во всех случаях есть прямое сравнение, поэтому эффективнее использовать хэш-индекс.

При добавлении индексов планы выполнения запросов изменятся, так как будет происходить индексный скан и Nested Loop станет быстрее благодаря индексам.

Результат EXPLAIN ANALYSE

```
Nested Loop Left Join (cost=0.57..136.37 rows=1 width=24) (actual time=26.272..26.785 rows=1 loops=1)
  -> Nested Loop (cost=0.28..128.07 rows=1 width=20) (actual time=0.021..0.533 rows=1 loops=1)
    -> Seq Scan on "Н_ОБУЧЕНИЯ" (cost=0.00..119.76 rows=1 width=4) (actual time=0.011..0.522 rows=1 loops=1)
      Filter: (("H3K")::text = '933232'::text)
      Rows Removed by Filter: 5020
    -> Index Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" (cost=0.28..8.30 rows=1 width=20) (actual time=0.007..0.007 rows=1 loops=1)
      Index Cond: (("ИД" = "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД") AND ("ИД" > 100012))
  -> Index Only Scan using "УЧЕН_ПК" on "Н_УЧЕНИКИ" (cost=0.29..8.30 rows=1 width=4) (actual time=26.249..26.250 rows=0 loops=1)
    Index Cond: ("ИД" = "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД")
    Heap Fetches: 0
Planning Time: 1.113 ms
Execution Time: 26.819 ms
```

Вывод

В ходе данной лабораторной работы я научился оптимизировать запросы, составлять их планы выполнения, а также выбирать подходящие индексы.